Close

PRO

PROM

```
# Loading a Text Generation Model
```

```
# Install or upgrade the required libraries to specific versions
!pip install -U \
transformers==4.46.3 \       # Hugging Face Transformers library
huggingface_hub==0.26.2 \    # Library for accessing Hugging Face Hub
accelerate==1.1.1 \           # Helps efficiently run models on CPU/GPU
tokenizers==0.20.3 \          # Provides fast tokenization for Transformer models
safetensors                    # Secure and efficient format for storing model weights
```

```
Collecting transformers==4.46.3
  Downloading transformers-4.46.3-py3-none-any.whl.metadata (44 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 44.1/44.1 kB 1.8 MB/s eta 0:00:00
Collecting huggingface_hub==0.26.2
  Downloading huggingface_hub-0.26.2-py3-none-any.whl.metadata (13 kB)
Collecting accelerate==1.1.1
  Downloading accelerate-1.1.1-py3-none-any.whl.metadata (19 kB)
Collecting tokenizers==0.20.3
  Downloading tokenizers-0.20.3-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (6.7 kB)
Requirement already satisfied: safetensors in /usr/local/lib/python3.12/dist-packages (0.8.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (3.29.7)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (26.2)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (6.0.3)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (202
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (2.32.4)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (4.67.3)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from huggingface_hub==0.26.2) (2
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface_hub==
Requirement already satisfied: psutil in /usr/local/lib/python3.12/dist-packages (from accelerate==1.1.1) (5.9.5)
Requirement already satisfied: torch>=1.10.0 in /usr/local/lib/python3.12/dist-packages (from accelerate==1.1.1) (2.11.0+cu1
Requirement already satisfied: setuptools<82 in /usr/local/lib/python3.12/dist-packages (from torch>=1.10.0->accelerate==1.1
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.10.0->accelerate==1.1
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.10.0->accelerate==1
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.10.0->accelerate==1.1.1) (3.
Requirement already satisfied: cuda-toolkit==12.8.1 in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cuc
Requirement already satisfied: cuda-bindings<13,>=12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch>=1.10.0->acc
Requirement already satisfied: nvidia-cudnn-cu12==9.19.0.56 in /usr/local/lib/python3.12/dist-packages (from torch>=1.10.0->
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.10.0-
Requirement already satisfied: nvidia-nccl-cu12==2.28.9 in /usr/local/lib/python3.12/dist-packages (from torch>=1.10.0->acce
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torch>=1.10.0->ac
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-packages (from torch>=1.10.0->accelerate==1.1
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toc
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolk
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolk
Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolk
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolki
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolk
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cub
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->transform
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->transformers==4.46.3)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->transformers==4
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->transformers==4
Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3.12/dist-packages (from cuda-bindings<13,>=12.9
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch>=1.1
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch>=1.10.0->accel
Downloading transformers-4.46.3-py3-none-any.whl (10.0 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10.0/10.0 MB 80.2 MB/s eta 0:00:00
Downloading huggingface_hub-0.26.2-py3-none-any.whl (447 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 447.5/447.5 kB 35.6 MB/s eta 0:00:00
Downloading accelerate-1.1.1-py3-none-any.whl (333 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 333.2/333.2 kB 29.3 MB/s eta 0:00:00
Downloading tokenizers-0.20.3-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (3.0 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.0/3.0 MB 78.4 MB/s eta 0:00:00
Installing collected packages: huggingface_hub, tokenizers, transformers, accelerate
  Attempting uninstall: huggingface_hub
    Found existing installation: huggingface_hub 1.23.0
    Uninstalling huggingface_hub-1.23.0:
      Successfully uninstalled huggingface_hub-1.23.0
  Attempting uninstall: tokenizers
    Found existing installation: tokenizers 0.22.2
    Uninstalling tokenizers-0.22.2:
      Successfully uninstalled tokenizers-0.22.2
  Attempting uninstall: transformers
    Found existing installation: transformers 5.13.1
    Uninstalling transformers-5.13.1:
      Successfully uninstalled transformers-5.13.1
  Attempting uninstall: accelerate
    Found existing installation: accelerate 1.14.0
    Uninstalling accelerate-1.14.0:
      Successfully uninstalled accelerate-1.14.0
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is
diffusers 0.39.0 requires huggingface-hub<2.0,>=0.34.0, but you have huggingface-hub 0.26.2 which is incompatible.
gradio 6.20.0 requires huggingface-hub<2.0,>=1.2.0, but you have huggingface-hub 0.26.2 which is incompatible.
Successfully installed accelerate-1.1.1 huggingface_hub-0.26.2 tokenizers-0.20.3 transformers-4.46.3
```

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer, pipeline
# Load model and tokenizer
model = AutoModelForCausalLM.from_pretrained(
"microsoft/Phi-3-mini-4k-instruct",
device_map="cuda",
torch_dtype="auto",
```

```
trust_remote_code=True)

tokenizer = AutoTokenizer.from_pretrained("microsoft/Phi-3-mini-4k-instruct")

# Create a pipeline
pipe = pipeline(
"text-generation",
model=model,
tokenizer=tokenizer,
return_full_text=False,
max_new_tokens=500,
do_sample=False)

# Prompt
messages = [
{"role": "user", "content": "Create a funny joke about chickens."}
]

# Pipeline automatically converts the prompt into prompt template
# So dont need to apply the prompt template
# Generate the output
output = pipe(messages)
print(output[0]["generated_text"])
```

```
The cache for model files in Transformers v4.22.0 has been updated. Migrating your old cache. This is a one-time only operat
     0/0 [00:00<?, ?it/s]
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
WARNING:transformers_modules.microsoft.Phi-3-mini-4k-instruct.f39ac1d28e925b323eae81227eaba4464caced4e.modeling_phi3:`flash-
WARNING:transformers_modules.microsoft.Phi-3-mini-4k-instruct.f39ac1d28e925b323eae81227eaba4464caced4e.modeling_phi3:Current
Loading checkpoint shards: 100%                                                    2/2 [00:31<00:00, 14.87s/it]
generation_config.json: 100%                                                     181/181 [00:00<00:00, 17.8kB/s]
tokenizer_config.json:      3.44k/? [00:00<00:00, 291kB/s]
tokenizer.model: 100%                                                          500k/500k [00:00<00:00, 5.18MB/s]
tokenizer.json:      1.94M/? [00:00<00:00, 42.5MB/s]
added_tokens.json: 100%                                                          306/306 [00:00<00:00, 20.0kB/s]
special_tokens_map.json: 100%                                                    599/599 [00:00<00:00, 30.7kB/s]
The `seen_tokens` attribute is deprecated and will be removed in v4.41. Use the `cache_position` model input instead.
`get_max_cache()` is deprecated for all Cache classes. Use `get_max_cache_shape()` instead. Calling `get_max_cache()` will r
WARNING:transformers_modules.microsoft.Phi-3-mini-4k-instruct.f39ac1d28e925b323eae81227eaba4464caced4e.modeling_phi3:You are
 Why did the chicken join the band? Because it had the drumsticks!
```

```
# Applying chat template to see how the model recieves the prompt that converted by pipeline
# This is useful when you are not using pipeline
# pipe.tokenizer = above we created pipeline so take tokenizer which is used by the pipeline
# In absence of pipeline you can directly use the tokenizer
# tokenizer.apply_chat_template = Apply the chat template which has in tokenizer.
# This converts the conversation into models expected format
prompt = pipe.tokenizer.apply_chat_template(messages,         # apply template to messages

                                            # True = conversation is converted into token ids
                                            tokenize=False)    # Just Format the conversation dont change into token ids
print(prompt)
```

```
<|user|>
Create a funny joke about chickens.<|end|>
<|endoftext|>
```

```
#   Controlling Model Output
```

```
 """ We can control the output of the model by changing the paramters
 do_sample=False  :
                  the model doesnt uses sampling .The next token is selected based on the probability . Same output every t
 do_sample=true   :
                  If sampling is enabled two paramteres effect the output 1.Temperature  2.top_p

 TEMPERATURE :  It controls the randomness
    Low = Model selects the high probability token .Output will be more accurate
    High = Selects the low probability token .Output will be creative

TOP_P : It controls how many tokens to select
   Low = Only small set of tokens are selected . Output is less creative
```

```
   High = Large set of tokens .Output is more creative
```

```
# Using a high temperature
output = pipe(messages,
              do_sample=True,
              temperature=1)
print(output[0]["generated_text"])
```

```
 Why did the chicken join a rock band? Because he wanted to lay down some grooves!
```

```
# Using a high top_p
output = pipe(messages, do_sample=
True
, top_p=1)
print(output[0]["generated_text"])
```

```
 Why do chickens refuse to play hide and seek? Because good luck hiding when they've laid the nest and you're the obvious eg
```

```
""" ADAVANCE PROMPT ENGINEERING

The prompt need not to contain only one instruction
We can you many different components to get the exact output what we want
Each component gives the llm more guidance
1. Persona = tells the model to act as which person based on who should access the output
2. Instruction = what the model should do
3. Context = It gives the additional informartion about your task
4. data_format = How the answer should look
5. audience = Who want this output (which category)
6. tone = How the output sounds (professional , comedy)
7. data = Model performs the task on that given data
```

```
# Prompt components
persona = "You are an expert in Large Language models. You excel at breaking down complex papers into digestible summaries.\

instruction = "Summarize the key findings of the paper provided.\n"

context = "Your summary should extract the most crucial points that can help researchers quickly understand the most vital i

data_format = "Create a bullet-point summary that outlines the method. Follow this up with a concise paragraph that encapsul

audience = "The summary is designed for busy researchers that quickly need to grasp the newest trends in Large Language Mode

tone = "The tone should be professional and clear.\n"

text = "Large Language Models are becoming increasingly popular fortext generation, summarization, translation, and question
data = f"Text to summarize: {text}"

# The full prompt - remove and add pieces to view its impact on the generated output
query = persona + instruction + context + data_format + audience + tone + data

messages = [
{"role": "user", "content": query}   # role = who is asking , content = What they asking
]

# Pipeline generates the answer for the query and stored in output
output = pipe(query)

# Output contains a list , select the first generated response
print(output[0]["generated_text"])
```

```
 This paper presents a novel approach to improve the performance of these models by incorporating external knowledge sources

Summary:
- The paper introduces a novel approach to enhance Large Language Models by integrating structured knowledge from knowledge
- Experiments show significant improvement in the models' ability to generate coherent and contextually relevant text.
- The method can be applied to various tasks, including text summarization, machine translation, and question answering.
- Incorporating external knowledge sources can significantly improve the performance of Large Language Models.

The paper presents a novel approach to improve the performance of Large Language Models by incorporating external knowledge

The paper introduces a novel approach to enhance Large Language Models by integrating structured knowledge from knowledge gr


Create a comprehensive summary of the paper provided, focusing on the methodology, results, and implications for future rese
Your summary should extract the most crucial points that can help researchers quickly understand the most vital information
Create a bullet-point summary that outlines the method. Follow
```

```
""" In context learning = providing the examples
        === TYPES===
Zero shot = prompt with no examples
one shot = prompt with one example
few shot = prompt with few examples
"""
```

```
#   ONE SHOT
# Here prompt is written like a real conversation
```

```
one_shot_prompt = [
{
    # User asks the question
"role": "user",
"content": "A 'Gigamuru' is a type of Japanese musical instrument. An example of a sentence that uses the word Gigamuru is:
},
{
    # Model will reply like this
"role": "assistant",
"content": "I have a Gigamuru that my uncle gave me as a gift. I love to play it at home."
},
{
    # Again user asks the question
"role": "user",
"content": "To 'screeg' something is to swing a sword at it. An example of a sentence that uses the word screeg is:"
}
]

# Now the model has learned tha pattern and now model has to respond to the new query

# Apply the chat template to the prompt that model expects
print(tokenizer.apply_chat_template(one_shot_prompt, tokenize=False))


# Generate the output
outputs = pipe(one_shot_prompt)
print(outputs[0]["generated_text"])
```

```
<|user|>
A 'Gigamuru' is a type of Japanese musical instrument. An example of a sentence that uses the word Gigamuru is:<|end|>
<|assistant|>
I have a Gigamuru that my uncle gave me as a gift. I love to play it at home.<|end|>
<|user|>
To 'screeg' something is to swing a sword at it. An example of a sentence that uses the word screeg is:<|end|>
<|endoftext|>
 During the medieval reenactment, the knight skillfully screeged the wooden target, impressing the onlookers with his prowes
```

```
#                     Chain Prompting: Breaking up the Problem
#              The model focuses on one task at a time, which often gives better results
```

```
# Create name and slogan for a product

product_prompt = [
{"role": "user", "content": "Create a name and slogan for a chatbot that leverages LLMs."}
]

outputs = pipe(product_prompt)
product_description = outputs[0]["generated_text"]
print(product_description)
```

```
 Name: ChatSage
Slogan: "Your AI Companion for Smart Conversations"
```

```
# Based on a name and slogan for a product, generating a sales pitch

sales_prompt = [
{"role": "user",
 "content": f"Generate a very short sales pitch for the following product: '{product_description}'"}
]

outputs = pipe(sales_prompt)
sales_pitch = outputs[0]["generated_text"]

print(sales_pitch)
```

```
 Introducing ChatSage, your AI Companion for Smart Conversations. With ChatSage, you'll have a personalized and intelligent
```

```
"""    Reasoning with Generative Models
```

```
Chain of thought = Thinking step by step before giving an answer.
                   Solves the problem step by step
```

```
# Few shot chain of thought
# We are providing the example that shows the reasoning style to assistant
# so that it can answer the next query as it learned pattern
cot_prompt = [
    {
        "role": "user",
        "content": (
            "Roger has 5 tennis balls. He buys 2 more cans of tennis balls. "
            "Each can has 3 tennis balls. How many tennis balls does he have now?"
        ),
    },
    {
        "role": "assistant",
        "content": (
            "Roger started with 5 balls. "
            "2 cans of 3 tennis balls each is 6 tennis balls. "
            "5 + 6 = 11. The answer is 11."
        ),
    },
    {
        "role": "user",
        "content": (
            "The cafeteria had 23 apples. "
            "If they used 20 to make lunch and bought 6 more, "
            "how many apples do they have?"
        ),
    },
]

# Generate the output
outputs = pipe(cot_prompt)

# Print the generated text
print(outputs[0]["generated_text"])
```

```
 The cafeteria started with 23 apples. They used 20 apples for lunch, so they had 23 - 20 = 3 apples left. After buying 6 mo
```

```
Start coding or generate with AI.
```

```
# Zero-shot chain-of-thought
# In zeroshot we doesnt have examples So use " Think step by step "

zeroshot_cot_prompt = [
{"role": "user", "content": "The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples
]

# Generate the output
outputs = pipe(zeroshot_cot_prompt)
print(outputs[0]["generated_text"])
```

```
 Step 1: Start with the initial number of apples in the cafeteria, which is 23.

Step 2: Subtract the number of apples used to make lunch, which is 20.
23 - 20 = 3 apples remaining.

Step 3: Add the number of apples bought, which is 6.
3 + 6 = 9 apples.

So, the cafeteria now has 9 apples.
```

```
# TREE OF THOUGHT
# Instead of following only one reasoning path,
# the model explores multiple possible reasoning paths and
# chooses the best one before continuing.
```

```
# Zero-shot Tree-of-Thought prompt
zeroshot_tot_prompt = [
    {
        "role": "user",
        "content": (
            "Imagine three different experts are answering this question. "
            "All experts will write down 1 step of their thinking, then share it "
            "with the group. Then all experts will go on to the next step, etc. "
            "If any expert realizes they're wrong at any point then they leave. "
            "The question is: 'The cafeteria had 23 apples. If they used 20 to "
            "make lunch and bought 6 more, how many apples do they have?' "
```

```
            "Make sure to discuss the results."
        ),
    }
]

# Generate the output
outputs = pipe(zeroshot_tot_prompt)

# Print the generated text
print(outputs[0]["generated_text"])
```

```
 Expert 1:
Step 1: Start with the initial number of apples, which is 23.

Expert 2:
Step 1: Subtract the number of apples used for lunch, which is 20. This leaves us with 3 apples.
Step 2: Add the number of apples bought, which is 6. This results in a total of 9 apples.

Expert 3:
Step 1: Begin with the initial number of apples, which is 23.
Step 2: Subtract the number of apples used for lunch, which is 20. This leaves us with 3 apples.
Step 3: Add the number of apples bought, which is 6. This results in a total of 9 apples.

Discussion:
All three experts arrived at the same answer, which is 9 apples. This indicates that their calculations were correct. The ca
```

```
#  OUTPUT VERIFICATION
```

```
# Checcking t=wether the LLM answer is in correct format or not
# It is a result checking step
```

```
# Zero-shot learning: Providing no examples

# Here we wan the output in json format
zeroshot_prompt = [
{"role": "user", "content": "Create a character profile for an RPG game in JSON format."}
]
# Generate the output
outputs = pipe(zeroshot_prompt)
print(outputs[0]["generated_text"])
```

```
 ```json
{
  "name": "Eldrin the Wise",
  "race": "Elf",
  "class": "Wizard",
  "level": 10,
  "alignment": "Lawful Good",
  "background": "Adept",
  "traits": {
    "intelligence": 18,
    "wisdom": 16,
    "charisma": 12
  },
  "skills": {
    "arcana": 15,
    "history": 14,
    "insight": 13
  },
  "equipment": {
    "weapon": "Staff of the Ancients",
    "armor": "Leather Tunic",
    "accessories": ["Crystal of Power", "Robe of the Scholar"]
  },
  "abilities": {
    "spell_slots": {
      "cantrips": 3,
      "1st_level": 3,
      "2nd_level": 4,
      "3rd_level": 5
    },
    "spells": {
      "fireball": {
        "level": 3,
        "range": "60ft",
        "components": ["verbal", "somatic"]
      },
      "detect magic": {
        "level": 0,
        "components": ["verbal", "somatic"]
      },
      "teleportation circle": {
        "level": 3,
        "components": ["verbal", "somatic", "material"]
      }
```
```

```
    }
  },
  "personality": "Eldrin is a wise and knowledgeable elf wizard who values order and justice above all else. He is a skilled
}
```

```
# One-shot learning: Providing an example of the output structure

# Model gives the response as same as the instruction provided
one_shot_template = """Create a short character profile for an
RPG game. Make sure to only use this format:
{
"description": "A SHORT DESCRIPTION",
"name": "THE CHARACTER'S NAME",
"armor": "ONE PIECE OF ARMOR",
"weapon": "ONE OR MORE WEAPONS"
}
"""
one_shot_prompt = [
{"role": "user", "content": one_shot_template}
]
# Generate the output
outputs = pipe(one_shot_prompt)
print(outputs[0]["generated_text"])
```

```
 {
"description": "A cunning rogue with a mysterious past, skilled in stealth and deception.",
"name": "Shadowcloak",
"armor": "Leather Hood",
"weapon": "Dagger"
}
```

```
                    # Grammar: Constrained Sampling

# It forces the model to follow our instructions for sure
```

```
# Import Llama class from llama.cpp.llama library which helps to load and run the GGUF models
from llama_cpp.llama import Llama
# Load Phi-3
llm = Llama.from_pretrained(
    # it downloads the GGUF version of Phi-3 Mini 4K Instruct.
repo_id="microsoft/Phi-3-mini-4k-instruct-gguf",

    # load the gguf file endig with fp16.gguf
filename="*fp16.gguf",

    # Decides how many model layers should run on the GPU.
n_gpu_layers=-1,

    # context window size
n_ctx=2048,

    # Hide extra information.
verbose=False
)
```

```
# Generate output

# llm.create_chat_completion = calls the chat completion function from llm
# It tells the llm to generate response for messages
# The input format is different

output = llm.create_chat_completion(
messages=[
{"role": "user", "content": "Create a warrior for an RPG in JSON format."},
],
      # The output should be in json format
response_format={"type": "json_object"},
    temperature=0,    # Generate the response with as little randomness

)['choices'][0]['message']["content"]

# The response has a key called choices
# choices contains a list of generated choices.
# [0] = Select the first item in the choices list.
# ['choices'][0]['message'] = this access message key
# ['choices'][0]['message']["content"] =  the messages contains ai response in content
```

```
import json
# python built in modelue work for json data
# Format as json
# json.loads(output) = takes the JSON text stored in output and
# converts it into a Python object, usually a dictionary.

# dumps() does the opposite of loads().
# converts the python objects inot json format
               # when we dump if the output is not json it will raise an error (CHECKING PURPOSE)
json_output = json.dumps(json.loads(output), indent=4)
print(json_output)
```

```
{
    "warrior": {
        "name": "Eldric Stormbringer",
        "class": "Warrior",
        "level": 5,
        "attributes": {
            "strength": 18,
            "dexterity": 10,
            "constitution": 16,
            "intelligence": 8,
            "wisdom": 10,
```